

National University of Computer and Emerging Sciences, Lahore Campus



Course:	COAL	Course Code:	EE2003
Program:	BS(CS,DS)	Semester:	Fall 2022
Duration:	3 Hours	Total Marks:	90
Paper Date:	16-12-2022	Page(s):	11
Section:	All	Roll No.	
Exam:	Final	Your Section:	

Instruction/Notes: This is an open notes/book exam. Sharing notes and calculators is **NOT ALLOWED**. All the answers should be written in provided space on this paper. Rough sheets can be used but will not be collected and checked. In case of any ambiguity, make reasonable assumptions. Questions during exams are not allowed.

Question 1 [Pipelining] [CLO 6] [10 Marks]:

Important Instruction: Following question is ONLY for section BSCS-A, BSCS-B, BSCS-F and BSCS-G

For the code segment given below, fill-in the following pipeline diagram. Clearly show the stall AND/OR forwarding where required. In case of forwarding, clearly draw the arrow and mention the name of operand that needs forwarding. Assume you have optimized pipelined MIPS Architecture (with all the hazards control implementation) as we have studied in class.

```
sub $1, $2, $3
or $4, $5, $6
lw $6, 100 ($4)
and $7, $6, $8
```

	CC1	CC2	CC3	CC4	CC5	CC6	CC7	CC8	CC9	CC10	CC11	CC12

Important Instruction: Following question is for ALL THE SECTIONS EXCEPT section BSCS-A, BSCS-B, BSCS-F and BSCS-G

Let us consider the following decomposition of the instruction processing

Fetch Instruction (FI): Read the next expected instruction into a buffer.

Decode Instruction (DI): Determine the opcode and the operand specifiers.

Fetch Operands (FO): Calculate the effective address of each source operand and fetch each operand from the memory. Operand in registers need not to be fetched.

Execute Instruction (EI): Perform the indicated operation and store the result if any, in the specified destination operand location.

Write Operand (WO): Store the result in memory.

Following is a set of instructions. Their implementation through pipelining has some data hazards. You have to solve those hazards by using stalling method.

Set of instructions

I1: mov bx, 0
I2: mov word [n1], ax
I3: add word [n1], bx
I4: add word [n1], 01
I5: mov cx, 0
I6: mov word [n2], cx
I7: add bx, [n1]
I8: add bx, word[n2]

Do it with Stalling Method (without Data Forwarding)

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
	FI	DI	FO	EI	WO															
		FI	DI	FO	EI	WO														
			FI	DI				FO	EI	WO										
				FI	DI						FO	EI	WO							
					FI	DI	FO	EI	WO											
						FI	DI		FO	EI	WO									
							FI	DI						FO	EI	WO				
								FI	DI							FO	EI	WO		

Question 2 [Cache] [CLO 6] [5+5 = 10 Marks]:

Consider a sequence of memory address references given below. In the sequence, each word address is provided in both the decimal and binary formats. Below each address, the relative time at which these references occur is also listed. Memory contents and addresses are shown in the second table.

Memory			Memory Access Sequence		
Decimal Address	Binary Address	Data	Time	Address Decimal	Address Binary
0	00 00 00 00	2	1	6	00 00 0 1 10
120	01 11 10 00	100	2	0	00 00 00 00
248	11 11 10 00	7	3	15	00 00 11 11
170	10 10 10 10	50	4	120	01 11 10 00
187	10 11 10 11	52	5	253	11 11 11 01
51	00 11 00 11	80	6	1	00 00 00 01
15	00 00 11 11	41	7	248	11 11 10 00
174	10 10 11 10	32	8	9	00 00 10 01
150	10 01 01 10	77	9	4	00 00 01 00
9	00 00 10 01	5	10	51	00 11 00 11
4	00 00 01 00	9	11	2	00 00 00 10
253	11 11 11 01	2	12	1	00 00 00 01
1	00 00 00 01	3			
7	00 00 01 11	65			
6	00 00 0 1 10	90			
2	00 00 00 10	55			

Now consider two different 8-word caches shown below. Assume that each of the caches was used independently to facilitate memory access for the sequence above. For each cache type, assume that the cache is initially empty. Assume that the least-recently used (LRU) scheme is used where appropriate. Also, when inserting an element into the cache, if there are multiple empty slots for one index, you should insert the new element into the left-most slot (first available slot).

Part (A) [5 Marks]: Use the direct-mapped cache to facilitate memory access for the memory sequence above. You should fill in the binary form of the Tag values. Show the final contents of the cache in the table below.

Note: V means Valid OR Value Bit.

Index	Direct-Mapped Cache		
	TAG	DATA	V
0	11111	7	0
1	00000	3	0
2	00000	55	0
3	00110	80	0
4	00000	9	0
5	11111	2	0
6	00000	90	0
7	00001	41	0

Hit Rate: 0

Miss Rate: 12

Part (B) [5 Marks]: Use the 2-way set associative cache to facilitate memory access for the memory sequence above. You should fill in the binary form of the Tag values. Show the final contents of the cache in the table below.

Set	2-way Set Associative Cache					
	V	Tag	Data	V	Tag	Data
0	0	111110	7	0	000001	9
1	0	000010	5	0	000000	3
2	0	000001	90	0	000000	55
3	0	000011	41	0	001100	80

Hit Rate: 1

Miss Rate: 11

Question 3 [Performance] [CLO 6] [10 Marks]: It takes 15 μ s to complete one instruction in a non-pipelined processor. We were able to convert the circuit to a 6 stage pipeline processor. Stage 1 to 6 take 2 μ s, 1.5 μ s, 3 μ s, 4 μ s, 1.5 μ s, 3 μ s resp. Time to move from one pipe stage to another is 2 μ s. (Note for Section BSCS-A, BSCS-B, BSCS-F and BSCS-G: Assume the transition time, to move from one pipe stage to another, is zero.)

Calculate the following values for pipeline and non-pipelined processor (Write the answer in the given table)

Value	Non-Pipeline	Pipeline
Clock Cycle	15 μ s	6 μ s
Clock Speed	0.067 Ms	0.167 Ms
Latency	15 μ s	36 μ s
Throughput for 46 instructions	0.067 Ms	0.15 Ms
Throughput for 1 instruction	0.067 Ms	0.027 Ms
Speedup of pipeline processor for 1 instruction	0.416s	
Speedup of pipeline processor for 75 instructions	2.345s	

Question 4 [Short Questions] [CLO 1,2,3,4,5] [5x6 = 30 Marks]:

- a. The following program is trying to add the first three numbers in the array num1 and store the sum in the fourth index of the num1 array. However, after running the program, the final sum generated is incorrect. **Identify mistakes in the program and write the correct code in the box on the right side.**

<pre> ; a program to add three numbers [org 0x0100] mov ax, [num1] mov bx, [num1+1] add ax, bx mov bx, [num1+2] add ax, bx mov [num1+3], ax mov ax, 0x4c00 int 0x21 num1: dw 5, 10, 15, 0 </pre>	<pre> ; Write Correct Code here [org 0x0100] mov ax, [num1] mov bx, [num1+2] add ax, bx mov bx, [num1+4] add ax, bx mov [num1+6], ax mov ax, 0x4c00 int 0x21 num1: dw 5, 10, 15, 0 </pre>
--	--

- b. Write a piece of code to check if a number 'num' is a power of two or not. If the number is power of two, set the PowerOfTwo flag to 1. You are only allowed to use shifting and logical instructions. You are not allowed to use DIV instruction.

<pre> [org 0x0100] jmp start PowerOfTwo :db 0 num: dw 0 start: mov ax, [num] push ax call CheckPowerOfTwo terminate: mov ax, 0x4c00 int 0x21 </pre>	<pre> CheckPowerOfTwo: push bp mov bp, sp push ax push bx mov ax, [bp+4] ;write your code here mov bx, ax sub bx, 1 AND ax, bx jnz l2 l1: mov byte[PowerOfTwo], 1 l2: pop bx pop ax pop bp ret 2 </pre>
--	---

- c. Following program has a function add1 that takes 2 numbers from stack and if sum of these two numbers is greater than 0, it returns their sum through stack otherwise it returns 0 through stack. Code has some logical errors. Highlight the errors and correct those errors so that you can pop the correct answer in the dx register. You can add or modify existing lines but you cannot remove any line.

<pre> jmp start add1: push bp mov bp, sp sub sp, 2 push ax mov ax, [bp+2] mov [bp-2], ax mov ax, [bp+4] add [bp+8], ax cmp word [bp+8], 0 ja end mov [bp+8], 0 end: pop bp ret 2 start: sub sp, 2 push 8 push 5 call add1 pop dx </pre>	<pre> add1: jmp start push bp mov bp, sp sub sp, 2 push ax mov ax, [bp+4] mov [bp+8], ax mov ax, [bp+6] add [bp+8], ax cmp word [bp+8], 0 jg end mov [bp+8], 0 end: pop ax mov sp, bp pop bp ret 4 start: sub sp, 2 push 8 push 5 call add1 pop dx </pre>
---	---

d. Answer the following questions:

(i) Which interrupt will be hooked after execution of following code?

68d/44h

```
[org 0x100]
;;; myISR is written here
xor ax,ax
mov es, ax
mov [es: 0x110] , myISR
mov [es : 0x112] , CS
mov ax, 4c00h
Int 21h
```

(ii) What is the total size (in bytes) of the interrupt vector table?

1KB

The first 22 words (in hex) of the 0th segment of the physical memory are shown in the following table (starting from data 0120).

(iii) What is the segment and offset of the interrupt service routine corresponding to interrupt 1?

Offset: 8002

Segment: 3F12

(iv) What is the segment and offset of the interrupt service routine corresponding to interrupt 5?

Offset: 06E1

Segment: AB56

(v) What is the segment and offset of the interrupt service routine corresponding to interrupt 10h?

Offset: Not in tab

Segment: Not in tab

0120
0140
0280
123F
124A
A198
BCD6
78D2
197B
CD79
E106
56AB
9851
0CDA
6502
AB69
F156
49D8
12E5
9857
146B
98A2

e. Write a piece of code that disables the timer interrupt in the PIC mask register.

```
[org 0x0100]
in al, 0x21
or al, 1
out 0x21, al
mov ax, 0x4c00
int 0x21
```

Question 5 [CLO 1,2,3,4,5] [30 Marks]: You are required to implement **Notepad** Application according to the functionality as described below:

- 1- Notepad application will start/load with following specifications:
 - a. Notepad will have two partitions in the display memory. Upper half (1st 12 rows) will be editor window while 2nd half (last 12 rows) will be read-only window. 13th row will be boundary line like "===== ". Both the editor and read-only windows will be space with white background initially. **[4 marks]**
 - b. A blinking cursor '|' (attribute: black on white background) will be on top-left cell of the Editor Window (1st half). For now, ignore the default cursor due to time constraint. **[4 marks]**
- 2- Editor window can be edited with following specifications (DO NOT take characters from user using software interrupts)
 - a. If user enters any key, that character or digit will be displayed at the position of cursor and cursor will move by one cell towards right. If cursor was at last cell of a row, it will move to 1st cell of next row within editor boundary. Due to time constraint, we assume that user will not write at bottom-right cell of Editor Window and beyond. You do not need to check this boundary condition. Assume that user will press only characters and numeric keys. Also assume that you are already given a function ScanCodeToAsciiConverter that reads scancode from AL and saves corresponding ascii in AH (You do not need to push or pop any parameter or return value and you do not need to re-write this converter, just use it wisely). **[6 marks]**
 - b. Make sure that your application doesn't write anything on Editor Window on key release. **[2 marks]**
 - c. Within Editor Window, user can move the cursor up, down, left or right by pressing Up, Down, Left, Right ARROW KEYS respectively. Due to time constraint we will implement only DOWN ARROW KEY (i.e. scancode 0x50). If user presses DOWN ARROW key, the cursor moves to same column of next row. Assume that user will not cross the Editor Window's boundary; you do not need to implement this boundary check. Make sure you properly handle previous position of cursor. This cursor movement is allowed even if there is no text written in editor window. **[6 marks]**
- 3- After every minute, Read-only window updates/refreshes itself with the latest content available on Editor Window i.e. after every minute, paste the content of Editor Window on Read-only Window (You do not need to remove cursor from read-only window). **[8 marks]**

Important Instructions:

- Credit will be given on code efficiency, so use string instructions where required.
- You may use the functions given in book examples. Give proper reference and use them wisely. Function calls should exactly support the required functionality.

<pre> ; Write your code here ; Data Declarations (if required) ; and Start/Main Functionality here jmp start oldisr: dd 0 cursorPosition: dw 0 buffer: times 4000 db 0 ; space for 4000 bytes tickcount: dw 0 message1: db 10, 13, 'Program started. \$' message2: db 10, 13, 'Message 22222... \$' message3: db 10, 13, 'Message 333... \$' exitFlag: db 0 start: xor ax, ax mov es, ax ; point es to IVT base mov ax, [es:9*4] mov [oldisr], ax ; save offset of old routine mov ax, [es:9*4+2] mov [oldisr+2], ax ; save segment of old routine cli ; disable interrupts mov word [es:9*4], kbisr ; store offset at n*4 mov [es:9*4+2], cs ; store segment at n*4+2 mov word [es:8*4], timer; store offset at n*4 mov [es:8*4+2], cs sti ; enable interrupts call clrscr jmp \$ l1: mov ah, 0 ; service 0 - get keystroke int 0x16 ; call BIOS keyboard service cmp al, 27 ; is the Esc key pressed jne l1 ; if no, check for next key mov ax, [oldisr] ; read old offset in ax mov bx, [oldisr+2] ; read old segment in bx cli ; disable interrupts mov [es:9*4], ax ; restore old offset from ax mov [es:9*4+2], bx ; restore old segment from bx sti ; enable interrupts mov ax, 0x4c00 ; terminate program int 0x21 </pre>	<pre> ;Write Timer Code here (if required) timer: push ax inc word [cs:tickcount]; increment tick count cmp word [cs:tickcount], 1080 je oneMinute jmp exitTimer oneMinute: mov word [cs:tickcount], 0 call saveScreen exitTimer: mov al, 0x20 out 0x20, al ; end of interrupt pop ax iret </pre>
--	--

<pre> ;Write KBISR here (if required) kbisr: push ax in al, 0x60 ; read a char from keyboard port test al, 0x80 jnz exit ; ignore release keys ; cmp al, 0xD0 ; DOWN Release ; je exit push es push di push ds push bx push cs pop ds mov bx, 0xb800 mov es, bx ; point es to video memory cmp al, 0x50 jne checkDown Up: mov di, [cursorPosition] mov byte[es:di], ' ' add di, 1 xor byte [es:di], 0x80 add di, 159 jmp UpdateCursor checkDown: mov byte[es:2], '*' mov di, [cursorPosition] mov byte [es:di], al ; yes, print R at top left UpdatePrevCursor: add di, 1 xor byte [es:di], 0x80 </pre>	<pre> ;Write KBISR here (if required) add di, 1 UpdateCursor: mov ah, 0xF0 mov al, ' ' mov word[es:di], ax mov [cursorPosition], di nomatch: pop bx pop ds pop di pop es pop ax jmp far [cs:oldisr] ; call the original ISR ; iret exit: pop ax jmp far [cs:oldisr] </pre>
--	---

<pre> ;Other Functions (if required) sleep: push cx mov cx, 0xFFFF delay: loop delay pop cx ret clrscr: push es push ax push di mov ax, 0xb800 mov es, ax mov di, 0 nextloc: mov word [es:di], 0x7020 add di, 2 cmp di, 1920 jne nextloc mov ah, 0x07 mov al, '=' nextloc2: mov word [es:di], ax ; clear next char on screen add di, 2 cmp di, 2080 jne nextloc2 nextloc3: mov word [es:di], 0x7020 add di, 2 cmp di, 4000 jne nextloc3 mov ah, 0xF0 mov al, ' ' mov word[es:0], ax pop di pop ax pop es ret </pre>	<pre> ;Other Functions (if required) saveScreen: pusha mov ax, 0xb800 mov ds, ax ; ds = 0xb800 mov es, ax mov cx, 1920 ; number of screen locations mov si, 0 mov di, 2080 cld ; set auto increment mode rep movsb ; save screen ;[es:di] = [ds:si] popa ret restoreScreen: pusha mov cx, 4000 ; number of screen locations mov ax, 0xb800 mov es, ax ; ds = 0xb800 push cs pop ds mov si, buffer mov di, 0 cld ; set auto increment mode rep movsb ; save screen ;[es:di] = [ds:si] popa ret </pre>
--	--

